Universidade Estadual de Mato Grosso do Sul

Curso de Ciência da Computação
Disciplina de Redes de Computadores
Trabalho de Redes de Computadores

Aplicação de serviços de cotação de preços de combustíveis

Felipe Echeverria Vilhalva

Dourados – MS

SUMÁRIO

1. **Introdução**

2. **Arquitetura do sistema**

   2.1. **UDP**

   2.2. **RDT 2.0**

3. **Implementação**

   3.1. **Cliente**

   3.2. **Servidor**

4. **Decisões do projeto**

5. **Tratamento da Retransmissão de Mensagens**

6. **Testes e resultados**

7. **conclusão**

8. **Referências**

1. Introdução

Este trabalho apresenta o desenvolvimento de uma aplicação cliente-servidor para um serviço de cotação de preços de combustíveis, utilizando a biblioteca Socket e o protocolo UDP. O principal desafio do projeto foi contornar a inerente falta de confiabilidade do UDP, o que foi solucionado com a implementação do protocolo de transferência de dados confiável RDT 2.0. Este mecanismo, baseado em confirmações (ACKs) e confirmações negativas (NAKs), garante a entrega íntegra das mensagens. A aplicação permite que clientes cadastrem informações de postos, que são salvas de forma persistente , e também realizem pesquisas para encontrar o menor preço de um combustível dentro de um raio de busca específico.

2. Arquitetura do Sistema

A aplicação foi estruturada no modelo cliente-servidor, permitindo a comunicação entre múltiplos clientes e um servidor central. A base da comunicação é o protocolo UDP, escolhido por sua simplicidade e baixa latência. Para superar a falta de confiabilidade do UDP, uma camada de controle de erros, o RDT 2.0, foi implementada. Este protocolo utiliza mensagens de confirmação (ACKs) e negação (NAKs) para garantir a entrega íntegra dos pacotes, formando a espinha dorsal da arquitetura de comunicação do sistema

2.1. UDP

O Protocolo de Datagramas do Usuário (UDP) é um protocolo de transporte simples e rápido, focado em velocidade em vez de confiabilidade. Ele opera no modelo "dispare e esqueça", enviando pacotes de dados (datagramas) sem estabelecer uma conexão prévia. Por não possuir mecanismos de confirmação, não há garantia de que os pacotes chegarão ao destino, nem que chegarão em ordem ou sem erros. Essa característica o torna ideal para aplicações em tempo real, como streaming de vídeo, jogos online e chamadas de voz, onde a

perda ocasional de pacotes é preferível a atrasos causados por retransmissões.

2.2. RDT 2.0

O RDT 2.0 é um protocolo projetado para criar uma camada de comunicação confiável sobre um canal que pode corromper pacotes, como o UDP. Sua principal característica é o uso de feedback do receptor para o emissor. O receptor analisa cada pacote recebido; se o pacote estiver intacto, ele envia uma mensagem de confirmação positiva (ACK - Acknowledgement). Se o pacote estiver corrompido, ele envia uma confirmação negativa (NAK - Negative Acknowledgement). Ao receber um NAK, o emissor simplesmente reenviará o último pacote, repetindo o processo até receber um ACK.

3. Implementação

A implementação foi realizada em C + +, com foco na modularidade e clareza. As trocas de mensagens são feitas através da serialização da “struct Pacote”, que encapsula todos os dados da comunicação, incluindo o mecanismo de erro simulado. O cliente foi implementado como uma Máquina de Estados Finitos para gerenciar o protocolo RDT 2.0 de forma robusta. O servidor, por sua vez, utiliza um modelo concorrente com múltiplas threads, permitindo o atendimento simultâneo de vários clientes de forma eficiente e segura.

3.1. Cliente

O cliente foi estruturado como uma Máquina de Estados Finitos (FSM) com três estados: um para aguardar a entrada do usuário, um para esperar a confirmação (ACK/NAK) e um terceiro para aguardar o resultado de pesquisas. Essa abordagem organiza a lógica de retransmissão por NAKs e timeouts de forma clara e previsível. A interação com o usuário é feita através de um menu de texto interativo no terminal, permitindo o envio de novos dados de postos ou a realização de consultas de preço.

Na figura 1, a “struct Pacote” é o tipo de dado fundamental para toda a comunicação de rede. Ela define o formato binário de cada mensagem trocada entre o cliente e o servidor, garantindo que ambos os lados interpretem os dados da mesma forma. Seus campos incluem:

- **Id**: Um identificador de sequência para associar respostas a requisições.
- **tipo_mensagem**: Um caractere que define a intenção do pacote ('D' para dados, 'P' para pesquisa, 'A' para ACK, 'N' para NAK, 'R' para resultado).

- **erro**: Um campo booleano utilizado pelo cliente para simular erros de transmissão, permitindo o teste do protocolo RDT 2.0.
- **union _tipo**: Uma union que otimiza o espaço, armazenando ou o preco (para mensagens 'D') ou o raio_de_busca (para mensagens 'P'), já que estes campos nunca são usados ao mesmo tempo.

```
struct Pacote {
  Pacote() : id(0), tipo_mensagem(' '), erro(false), tipo_combustivel(0), latitude(0.0), longitude(0.0) {
      _tipo.preco = 0;
  }

  int id;
  char tipo_mensagem;       // 'D' ou 'P' ou 'A' ou 'N' para ACK e NAK
  bool erro;                // true ou false
  int tipo_combustivel;     // 0-diesel, 1-álcool, 2-gasolina

  union {
    int preco;              // para tipo 'D'
    int raio_de_busca;      // para tipo 'P'
  } _tipo; // _tipo

  double latitude;
  double longitude;
};
```

Figura 1 - Estrutura do pacote

Na figura 2 temos um tipo enumerado que é o pilar da Máquina de Estados Finitos (FSM) no lado do cliente, responsável por gerenciar o protocolo de transferência de dados confiável. Ele define os três estados possíveis em que o cliente pode se encontrar:

1. **AGUARDANDO_DADOS**: O estado inicial, onde o cliente está pronto para aceitar uma nova entrada do usuário.
2. **AGUARDANDO_CONFIRMACAO**: O estado de espera após o envio de um pacote, aguardando um ACK ou NAK do servidor.
3. **AGUARDANDO_RESULTADO_PESQUISA**: Um estado específico para quando o cliente enviou uma pesquisa ('P') e, após receber o ACK, aguarda um segundo pacote com o resultado.

```
enum Estado {
    AGUARDANDO_DADOS,
    AGUARDANDO_CONFIRMACAO,
    AGUARDANDO_RESULTADO_PESQUISA
};
```

Figura 2 - Enumeração da máquina de estados

3.2. Servidor

O servidor foi projetado para ser concorrente, utilizando a biblioteca Pthreads. A thread principal atua como um despachante, recebendo todos os pacotes UDP e criando uma nova thread para cada requisição. Isso permite que múltiplos clientes sejam atendidos simultaneamente. Para garantir a integridade dos dados compartilhados (o vetor de postos e o arquivo “postos.txt”), foi implementado um “mutex”. Este mecanismo de exclusão mútua cria uma seção crítica, garantindo que apenas uma thread possa modificar os dados por vez, evitando condições de corrida.

A classe “Posto”, como apresentado na figura 3, é a representação de um posto de combustível no sistema. Ela agrupa todos os dados pertinentes a um posto: seu “preco” para um “tipoCombustivel” específico e sua localização geográfica (latitude e longitude). No servidor, objetos desta classe são armazenados em um vetor em memória para permitir buscas rápidas e também são serializados para um arquivo de texto para garantir a persistência dos dados entre as execuções do programa.

```
class Posto{
public:
  Posto() : preco(0), tipoCombustivel(0), latitude(0.0), longitude(0.0) {} // Definição do construtor

  int preco;
  int tipoCombustivel;
  double latitude;
  double longitude;
};

vector<Posto> postos; // vetor que armazena os postos sem precisar ficar abrindo o arquivo (ponto critico)
```

Figura 3 - Classe do posto para armazenar os postos na máquina

Na figura 4, a “struct request_data” atua como um contêiner para transportar as informações de uma requisição da thread principal do servidor para a nova thread de processamento. Ela encapsula três informações essenciais:

- O descritor do **socket** principal, para que a thread possa enviar respostas.
- A estrutura **cliente_addr**, contendo o endereço IP e a porta do cliente, para que a resposta seja enviada para o destino correto.
- O **pacote_recebido**, uma cópia do pacote enviado pelo cliente, que a thread irá processar.

```c
typedef struct {
    int socket;
    struct sockaddr_in cliente_addr;
    Pacote pacote_recebido;
} request_data;
```

Figura 4 - Estrutura para passar múltiplos valores em cada thread

Para atender às requisições de pesquisa por raio, foi implementada uma lógica para calcular a distância geodésica entre dois pontos como demonstrado na figura 5.

- **calcularDistancia**: Esta função implementa a **Fórmula de Haversine**, um método matemático preciso para calcular a distância em uma esfera a partir de latitudes e longitudes. Ela foi escolhida por levar em conta a curvatura da Terra, garantindo a correção do filtro por raio.
- **paraRadianos**: Uma pequena função auxiliar que converte ângulos de graus para radianos, um pré-requisito para o uso das funções trigonométricas da biblioteca <cmath>.

```c
double paraRadianos(double graus) {
    return graus * M_PI / 180.0;
}

double calcularDistancia(double lat1, double lon1, double lat2, double lon2) {
    double dLat = paraRadianos(lat2 - lat1);
    double dLon = paraRadianos(lon2 - lon1);

    double radLat1 = paraRadianos(lat1);
    double radLat2 = paraRadianos(lat2);

    double a = sin(dLat / 2) * sin(dLat / 2) +
               cos(radLat1) * cos(radLat2) *
               sin(dLon / 2) * sin(dLon / 2);
    double c = 2 * atan2(sqrt(a), sqrt(1 - a));

    return RAIO_TERRA * c;
}
```

Figura 5 - Funções para calcular a distância entre duas coordenadas

4. Decisões de Projeto

Durante o desenvolvimento, algumas decisões de implementação foram tomadas para funcionalidades não detalhadas na especificação. A escolha por um modelo multithread no servidor visou a eficiência e a facilidade no compartilhamento de memória. Para a persistência de dados, optou-se por um arquivo de texto de formato simples, pela facilidade de depuração. A interface do cliente foi projetada com um menu interativo para facilitar o uso e a demonstração de todas as funcionalidades. Por fim, para o cálculo de distância entre coordenadas geográficas, foi implementada a Fórmula de **Haversine** por sua precisão ao considerar a curvatura da Terra.

5. Tratamento da Retransmissão de Mensagens

A transmissão de dados confiável sobre o protocolo não confiável UDP foi o principal desafio técnico deste trabalho. Para solucionar a possibilidade de pacotes corrompidos ou perdidos, foi implementado o protocolo RDT 2.0, cujo mecanismo de retransmissão foi gerenciado por uma Máquina de Estados Finitos (FSM) no lado do cliente.

Máquina de Estados Finitos (FSM) do Cliente

O cliente opera com base em uma FSM de três estados, que dita seu comportamento em cada etapa da comunicação:

1. **AGUARDANDO_DADOS**: Neste estado, o cliente está ocioso e pronto para receber uma nova requisição do usuário (seja de cadastro 'D' ou pesquisa 'P'). Ao receber os dados, ele monta e envia o pacote, transitando imediatamente para o estado de espera.
2. **AGUARDANDO_CONFIRMACAO**: Após enviar um pacote, o cliente entra neste estado crucial. Ele ativa um **timeout** de 2 segundos no socket e aguarda uma resposta do servidor. A retransmissão do pacote original ocorre sob três condições:
    - **Recebimento de um NAK**: Indica que o servidor recebeu o pacote, mas ele estava corrompido.
    - **Ocorrência de Timeout**: Indica que o pacote original ou a resposta do servidor foi perdida na rede.
    - **Recebimento de um pacote inesperado**: Protege contra a reordenação de pacotes, ignorando respostas que não sejam um ACK ou NAK.
3. **AGUARDANDO_RESULTADO_PESQUISA**: Um estado específico para o fluxo de pesquisa ('P'). Após receber um ACK, o cliente transita para este estado, onde aguarda um segundo pacote do servidor contendo o resultado da busca.

Lógica do Servidor

O servidor possui uma lógica mais simples. Ele responde a cada pacote individualmente: envia um **ACK** para pacotes válidos e um **NAK** para pacotes que chegam com a flag de erro simulado ativada. Essa resposta direta é o que alimenta a lógica de decisão da FSM do cliente, permitindo que o cliente saiba se a transmissão foi bem-sucedida ou se precisa ser repetida.

6. Testes e resultados

Para validar a aplicação, foi executada uma série de testes que abrangeram todas as funcionalidades principais. Foram testados o envio de dados ('D') e a pesquisa ('P'), incluindo a verificação da lógica de seleção do menor preço e o filtro por raio. O protocolo de confiabilidade RDT 2.0 foi validado através da simulação de erros, que demonstrou o correto funcionamento dos mecanismos de NAK e retransmissão por timeout. Os testes de concorrência, com múltiplos clientes, confirmaram a robustez do servidor. As capturas de tela apresentadas comprovam os resultados bem-sucedidos.

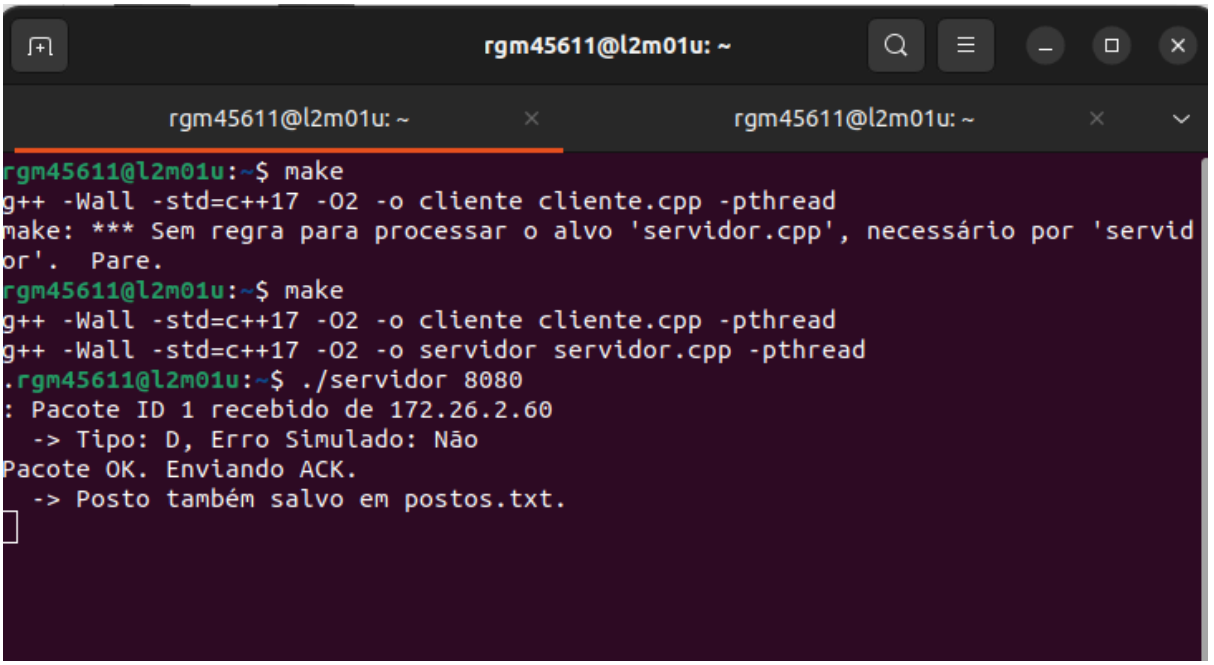

Figura 6 - Servidor na máquina l2m01u

```
rgm45611@l2m02u:~$ ./cliente 172.26.2.101 8080
D 1 0 5999 -22.2319 -54.5070
[EVENTO] ACK recebido para o pacote ID 1. Sucesso na confirmação!
```

Figura 7 - Cliente 1 na máquina l2m02u inserindo dados

```
→ T1 git:(main) ✗ ./cliente 172.26.2.101 8080
P 1 2 1000 -22.2218 -54.8064
[EVENTO] NAK recebido. Retransmitindo pacote ID 1
[EVENTO] ACK recebido para o pacote ID 1. Sucesso na confirmação!
----------RESULTADO DA PESQUISA -----------
Menor preço encontrado: R$ 5.8900
Localização: Lat -22.2250, Lon -54.8152
```

Figura 8 - Cliente 2 na minha máquina(notebook)

```
P 1 2 5000 -22.2218 -54.8064
[EVENTO] NAK recebido. Retransmitindo pacote ID 1
[EVENTO] NAK recebido. Retransmitindo pacote ID 1
[EVENTO] NAK recebido. Retransmitindo pacote ID 1
[EVENTO] NAK recebido. Retransmitindo pacote ID 1
[EVENTO] ACK recebido para o pacote ID 1. Sucesso na confirmação!
----------- RESULTADO DA PESQUISA -----------
Menor preço encontrado: R$ 5.8500
Localização: Lat -22.2198, Lon -54.8207
-----------------------------------------
```

Figura 9 - Cliente 1 pesquisando e enviando o pacote com erro

```
: Pacote ID 1 recebido de 172.26.2.60
  -> Tipo: P, Erro Simulado: Sim
: Detectado erro no pacote. Enviando NAK.
: Pacote ID 1 recebido de 172.26.2.60
  -> Tipo: P, Erro Simulado: Sim
: Detectado erro no pacote. Enviando NAK.
: Pacote ID 1 recebido de 172.26.2.60
  -> Tipo: P, Erro Simulado: Sim
: Detectado erro no pacote. Enviando NAK.
: Pacote ID 1 recebido de 172.26.2.60
  -> Tipo: P, Erro Simulado: Sim
: Detectado erro no pacote. Enviando NAK.
: Pacote ID 1 recebido de 172.26.2.60
  -> Tipo: P, Erro Simulado: Não
Pacote OK. Enviando ACK.
  -> Iniciando pesquisa de preço...
  -> Posto mais barato encontrado! Preço: 5850
  -> Pacote de resultado da pesquisa enviado.
```

Figura 10 - Servidor processando erro.

O conjunto de testes foi realizado no laboratório 2 do curso de ciência da computação/sistemas de informação.

7. Conclusão

O desenvolvimento deste projeto permitiu a aplicação prática de conceitos fundamentais de redes de computadores. A implementação bem-sucedida de um protocolo de transporte confiável (RDT 2.0) sobre o UDP, juntamente com um servidor concorrente, demonstrou o cumprimento de todos os objetivos propostos. O trabalho solidificou o conhecimento sobre programação de sockets, controle de concorrência com threads e mutexes, e a complexidade envolvida na criação de protocolos de comunicação robustos. O sistema funciona conforme especificado e serve como uma base sólida para futuras expansões.

8. Referências

GNU make
A Simple Makefile Tutorial
Haversine formula to find distance between two points on a sphere - GeeksforGeeks